

Hamiltonian Neural Network for Pseudo Marginal Hamiltonian Monte Carlo

Yeda CUI

February 2, 2025

This report outlines my attempt to apply the Hamiltonian Neural Network (HNN) to the Pseudo Marginal HMC (PMHMC) method. Section 1 provides a recap of the PMHMC method introduced by Alenlöv et al. (2021). Section 2 details my approach to developing an HNN-based method for PMHMC. Finally, Section 3 highlights financial models that can benefit from either HNN or PMHMC. All notations are consistent with those in Alenlöv et al. (2021), unless otherwise specified.

1 Background

Alenlöv et al. (2021) proposed PMHMC based on the extended Hamiltonian system defined in equation (10). Since the integration error increases with the dimensionality, they split the extended Hamiltonian H into two components, A and B , as follows:

$$A(\rho, \mathbf{u}, \mathbf{p}) := \frac{1}{2} \left\{ \rho^\top \rho + \mathbf{u}^\top \mathbf{u} + \mathbf{p}^\top \mathbf{p} \right\}, \quad B(\theta, \mathbf{u}) := -\log p(\theta) - \log \hat{p}(y_{1:T} \mid \theta, \mathbf{u}),$$

where both components have analytical solutions. The solutions of these two systems after h units of time, denoted as Φ_h^A and Φ_h^B , are as follows:

$$\Phi_h^A : \begin{cases} \theta(h) = \theta(0) + h\rho(0) \\ \rho(h) = \rho(0) \\ \mathbf{u}(h) = \mathbf{p}(0) \sin(h) + \mathbf{u}(0) \cos(h) \\ \mathbf{p}(h) = \mathbf{p}(0) \cos(h) - \mathbf{u}(0) \sin(h) \end{cases}$$

and

$$\Phi_h^B : \begin{cases} \theta(h) = \theta(0) \\ \rho(h) = \rho(0) + h \nabla_\theta \{ \log p(\theta) + \log \hat{p}(y_{1:T} \mid \theta, \mathbf{u}(0)) \}_{|\theta=\theta(0)} \\ \mathbf{u}(h) = \mathbf{u}(0) \\ \mathbf{p}(h) = \mathbf{p}(0) + h \nabla_{\mathbf{u}} \log \hat{p}(y_{1:T} \mid \theta(0), \mathbf{u})_{|\mathbf{u}=\mathbf{u}(0)} \end{cases}. \quad (1)$$

The numerical integration of the whole Hamiltonian system after hL units of time is given by

$$\hat{\Phi}_{hL} = \Phi_{h/2}^A \circ \{ \Phi_h^B \circ \Phi_h^A \}^{L-1} \circ \Phi_h^B \circ \Phi_{h/2}^A.$$

Sampling under Generalized Linear Mixed Model I implemented the PMHMC method introduced in Alenlöv et al. (2021). While replicating the results from Section 4.3 and using the same prior and initial marginal state θ , my sampled values for μ_1 and μ_2 did not concentrate in the correct region. To address this, I adopted a narrower prior, specifically $N(0, 1)$, for all θ . Additionally, the initial states for $\log \lambda_1$, $\log \lambda_2$, and $\text{logit } w_1$ were set to zero. Under these modified settings, the sampling results appeared much more reasonable, as illustrated in Figure 1.

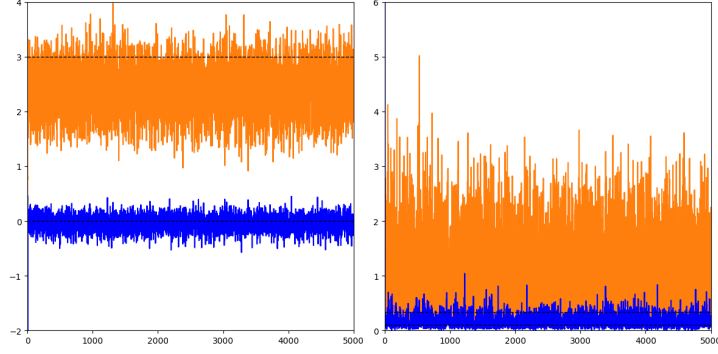


Figure 1: Traces for parameters μ_1 and μ_2 (left) and $1/\lambda_1$ and $1/\lambda_2$ (right) for the pseudo-marginal HMC sampler with $N = 128$.

2 Hamiltonian Neural Network for PMHMC

Based on my understanding, since the Hamiltonian system A is explicit and has an analytical solution, it's only necessary to use an HNN to learn the Hamiltonian system B and replace the numerical partial derivatives in Equation (1). For some network parameters ϕ , let $H_\phi(\theta, \rho, \mathbf{u}, \mathbf{p})$ represents the HNN's prediction of the Hamiltonian system B. To train the HNN, we can adopt the same loss function proposed in Greydanus et al. (2019) and Dhulipala et al. (2023). Given training data $\{\theta, \rho, \mathbf{u}, \mathbf{p}\}$, numerical gradients are first computed to give the numerical time derivatives $\left\{ \frac{d\theta}{dt}, \frac{d\rho}{dt}, \frac{d\mathbf{u}}{dt}, \frac{d\mathbf{p}}{dt} \right\}$ as follows:

$$\begin{aligned} \frac{d\theta}{dt} &= 0, & \frac{d\rho}{dt} &= \nabla_\theta \{ \log p(\theta) + \log \hat{p}(y_{1:T} | \theta, \mathbf{u}(0)) \}_{|\theta=\theta(0)}, \\ \frac{d\mathbf{u}}{dt} &= 0, & \frac{d\mathbf{p}}{dt} &= \nabla_{\mathbf{u}} \log \hat{p}(y_{1:T} | \theta(0), \mathbf{u})_{|\mathbf{u}=\mathbf{u}(0)}. \end{aligned}$$

HNNs optimize the network parameters ϕ to minimize the following loss function:

$$L = \left\| \frac{\partial H_\phi}{\partial \rho} - \frac{d\rho}{dt} \right\| + \left\| -\frac{\partial H_\phi}{\partial \theta} - \frac{d\theta}{dt} \right\| + \left\| \frac{\partial H_\phi}{\partial \mathbf{p}} - \frac{d\mathbf{p}}{dt} \right\| + \left\| -\frac{\partial H_\phi}{\partial \mathbf{u}} - \frac{d\mathbf{u}}{dt} \right\|.$$

After training the neural network, the integration solution for the Hamiltonian system B can be approximated as:

$$\tilde{\Phi}_h^B : \begin{cases} \theta(h) = \theta(0) \\ \rho(h) = \rho(0) - h \frac{\partial H_\phi}{\partial \theta}(\theta(0), \rho(0), \mathbf{u}(0), \mathbf{p}(0)) \\ \mathbf{u}(h) = \mathbf{u}(0) \\ \mathbf{p}(h) = \mathbf{p}(0) - h \frac{\partial H_\phi}{\partial \mathbf{p}}(\theta(0), \rho(0), \mathbf{u}(0), \mathbf{p}(0)) \end{cases}.$$

The numerical integration of the whole Hamiltonian system after hL units of time can be approximated by

$$\tilde{\Phi}_{hL} = \Phi_{h/2}^A \circ \left\{ \tilde{\Phi}_h^B \circ \Phi_h^A \right\}^{L-1} \circ \tilde{\Phi}_h^B \circ \Phi_{h/2}^A. \quad (2)$$

2.1 PMHMC by L-HNN in Dhulipala et al. (2023)

Firstly, I attempted to directly adopt the L-HNN structure proposed in Dhulipala et al. (2023), where H_ϕ is represented as a feedforward neural network. The input dimension of the network is $2d + 2D$. For the generalized linear mixed model example discussed in Section 4.3 of Alenlöv et al. (2021), $d = 13$ and $D = 64,000$, resulting in a neural network input dimension of 128,026

which is dramatically higher than the examples in Dhulipala et al. (2023), where the highest input dimension is only 100. Both the input and output dimensions in the training dataset are also high-dimensional.

Unlike Dhulipala et al. (2023), where the entire dataset can be loaded into memory for training, this is not feasible here due to memory limitations. Instead, I loaded batches of data into memory for training and utilized the TFRecord data format to improve I/O efficiency, as opposed to the pickle format used in Dhulipala et al. (2023). For training, I used a batch size of 512, a learning rate of 0.001, and 10,000 training steps. However, the training mean squared error loss only decreased to 0.01. Figure 2 compares the gradient of the Hamiltonian with respect to the marginal variables θ , computed via autograd and the predictions of the HNN. Figure 2a shows that the partial derivatives computed by the LHNN are significantly incorrect. Additionally, Figure 2b demonstrates that the Hamiltonian cannot be conserved during integration using Equation (2). The acceptance rate using LHNNs is nearly zero. I also tested different learning rates $\{0.1, 0.01, 0.0001\}$ and neural network widths 500, 2000, but the results remained similar.

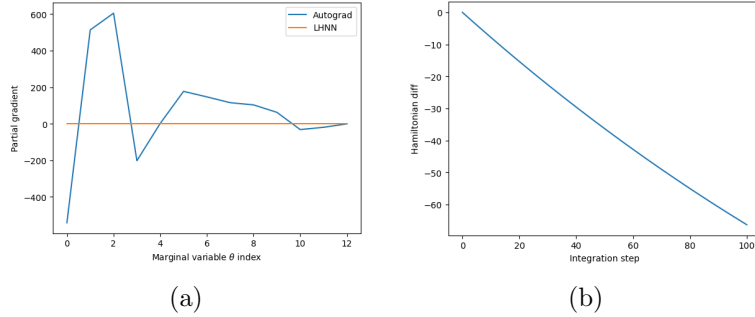


Figure 2: (a) The partial derivatives of the 13 marginal parameters computed by autograd and LHNN. (b) The Hamiltonian path after 100 steps of integration using LHNN via Equation (2).

To investigate why the loss cannot be further reduced, I speculate that the issue arises from the high dimensionality of the problem, which limits the LHNN’s ability to effectively represent the Hamiltonian system. To test this hypothesis, I used the LHNN to learn the Hamiltonian of standard normal density for a total of only 16 samples under varying dimensions. The setup involved a three-layer neural network with a width of 500, trained over 100 epochs, with 1000 training steps per epoch. The mean squared error (MSE) loss paths during training are shown in Figure 3. From this figure, we observe that when the dimension is as high as 25,000, the LHNN struggles to train effectively and fails to learn the Hamiltonian gradients of the 16 standard normal samples. This suggests that the learning capacity of the LHNN under high dimensions is limited. It is important to note that this test involves a relatively simple task—learning the gradients of the Hamiltonian for only 16 samples drawn from a standard normal distribution. For more complex tasks, such as learning the Hamiltonian for the general linear mixed model, more sophisticated architectures are likely required.

Advantage with HNN One advantage of the HNN approach is that the simulation process is significantly faster. For example, in the generalized linear mixed model discussed in Section 4.3 of Alenlöv et al. (2021), there are a total of 3000 observations, making the calculation of gradients for the Hamiltonian system highly time-consuming. In contrast, computing the gradient using the HNN is considerably more efficient.

2.2 Modifications to the Structure of HNN for PMHMC

Due to the high dimensionality of the problem, I proposed a modification to the structure of the HNN to reduce the input dimension of the neural network. It is observed that the Hamiltonian

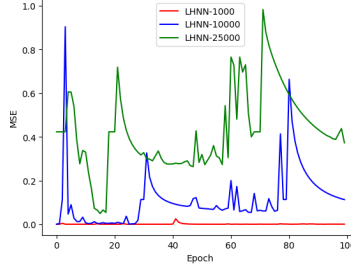


Figure 3: The MSE loss path for learning the Hamiltonian gradients of standard normal density with varying dimensions.

B can be expressed as the summation of simple functions with different inputs:

$$B = -\log p(\theta) - \log \hat{p}(y_{1:T} \mid \theta, \mathbf{U}) = -\log p(\theta) - \sum_{k=1}^T \log \hat{p}(y_k \mid \theta, \mathbf{U}_k).$$

We can utilize a neural network $\mathcal{H}_\phi(y, \theta, u)$ to approximate the function $-\log \hat{p}(y \mid \theta, u)$. The HNN for predicting B is then defined as:

$$H_\phi(\theta, \rho, \mathbf{u}, \mathbf{p}) = \frac{1}{2} \theta^T \theta + \sum_{k=1}^T \mathcal{H}_\phi(y_k, \theta, \mathbf{u}_k),$$

where the prior of θ is assumed to be a normal distribution. I implemented this idea; however, the results are still unsatisfactory and the acceptance rate is still close to zero.

3 Beneficial Financial Models by [Alenlöv et al. \(2021\)](#) or [Dhulipala et al. \(2023\)](#)

Stochastic Volatility Model

The stochastic volatility (SV) model is a widely used framework in financial econometrics for modeling time-varying volatility in financial time series. It assumes that the observed series is driven by a latent (unobservable) volatility process that evolves stochastically over time. The general form of the SV model is given by:

$$\begin{aligned} dS_t &= \mu S_t dt + \sqrt{\nu_t} S_t dW_t, \\ d\nu_t &= \alpha_{\nu,t} dt + \beta_{\nu,t} dB_t, \end{aligned}$$

where S_t is the stock price process and ν_t is the volatility process. While S_t can be observed, ν_t is latent and unobservable. If we perform posterior estimation of both the model parameters and ν_t , several challenges arise: The Metropolis-Hastings algorithm faces difficulties in selecting a proper proposal density for the latent volatility process ν_t . The traditional HMC method struggles when the posterior distribution of ν_t exhibits strong multimodality. For example, the posterior distribution of ν_t under the Heston model shows strong multimodes, as shown in Section 2.3 of [Cape et al. \(2015\)](#). The PMHMC method can overcome the multimodality issue, making it a suitable approach for such problems.

Remark 1. Similarly, the posterior of the latent regime variable in regime-switching models also exhibits multimodality issues, making it a good candidate for benefiting from PMHMC. Other models with some variables that exhibit multimodality, but where we are not concerned with their distribution, can also leverage PMHMC effectively.

Credit Risk Modeling with Large Loan Portfolios Credit risk modeling is crucial for financial institutions to estimate the probability that borrowers will default on their loan obligations. In the logistic regression model, the probability of default for borrower i is modeled as:

$$P(Y_i = 1 \mid \mathbf{X}_i) = \frac{1}{1 + e^{-(\beta_0 + \mathbf{X}_i^\top \boldsymbol{\beta})}},$$

where Y_i is a binary variable indicating default ($Y_i = 1$) or non-default ($Y_i = 0$), $\mathbf{X}_i$ is a vector of predictor variables for borrower i , and $\boldsymbol{\beta}$ is a vector of coefficients corresponding to the predictor variables.

Although $\boldsymbol{\beta}$ can be estimated using regression techniques, obtaining the posterior distribution of $\boldsymbol{\beta}$ is often desirable. The posterior distribution allows for the incorporation of prior information and enables robust risk analysis, such as assessing tail risk or calculating Conditional Value-at-Risk (CVaR) based on $\boldsymbol{\beta}$.

When dealing with large datasets, such as loan portfolios with a vast number of observations, traditional HMC methods can become computationally expensive and inefficient. This is due to the high cost of computing gradients for the entire dataset during each sampling iteration. However, the HNN approach introduced in [Dhulipala et al. \(2023\)](#) provides a significantly more efficient method for sampling from the posterior distribution. The HNN framework leverages second-order information, allowing for faster convergence and improved computational efficiency, making it suitable for large-scale credit risk modeling.

Remark 2. Similarly, other models with a large number of observations, where traditional HMC becomes computationally expensive, can also benefit from the efficient sampling capabilities of HNN.

Remark 3 (Testing). All the source code is located in the `./src/` folder, with each Python script having a corresponding test file in the `./tests/` folder to validate all the classes and functions.

References

- Alenlöv, J., A. Doucet, and F. Lindsten (2021). Pseudo-marginal hamiltonian monte carlo. *Journal of Machine Learning Research* 22(141), 1–45.
- Cape, J., W. Dearden, W. Gamber, J. Liebner, Q. Lu, and M. L. Nguyen (2015). Estimating heston’s and bates’ models parameters using markov chain monte carlo simulation. *Journal of Statistical Computation and Simulation* 85(11), 2295–2314.
- Dhulipala, S. L., Y. Che, and M. D. Shields (2023). Efficient bayesian inference with latent hamiltonian neural networks in no-u-turn sampling. *Journal of Computational Physics* 492, 112425.
- Greydanus, S., M. Dzamba, and J. Yosinski (2019). Hamiltonian neural networks. *Advances in neural information processing systems* 32.